LAB REPORT: BINOMIAL & FIBONACCI HEAP OPERATIONS & ANALYSIS

Course: Design & Analysis of Algorithms

Name: Aryan Nautiyal **Roll Number:** 251210015

Aim

To implement insertion, find-minimum, and deletion (extract-min) operations in a Binomial Heap and a Fibonacci Heap, display the structure changes after each operation, and record the execution time of insertion and deletion for different values of n.

Theory

Operation	Binomial (Worst)	Binomial (Amortised)	Fibonacci (Worst)	Fibonacci (Amortised)
Insert	$O(\log n)$	$O(1)$	$O(1)$	$O(1)$
Find-Min	$O(\log n)$	$O(\log n)$	$O(1)$	$O(1)$
Extract-Min	$O(\log n)$	$O(\log n)$	$O(n)$	$O(\log n)$
Union/Merge	$O(\log n)$	$O(\log n)$	$O(1)$	$O(1)$
Decrease-Key	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$

- **Binomial Heap:** A collection of binomial trees satisfying the min-heap property. A binomial tree B_k has exactly 2^k nodes and height k. A binomial heap on n nodes contains at most $\lfloor \log_2 n \rfloor + 1$ trees, corresponding to the set bits of n. Insertion creates a B_0 and merges the root list, mirroring binary addition.
- **Fibonacci Heap:** A collection of min-heap-ordered trees with lazy consolidation. Insertion simply appends the new node to the root list in $O(1)$. Trees are consolidated only during extract-min, bounded by $D(n) = O(\log n)$ using the golden-ratio argument. Each node also carries a **mark** bit used by decrease-key / cascading cut.
- **Key Difference:** Binomial heaps consolidate eagerly on every insert; Fibonacci heaps defer all restructuring to extract-min. This makes Fibonacci heaps theoretically superior for algorithms with many decrease-key operations (Dijkstra, Prim), but the large constant factor often makes binomial / binary heaps faster in practice.

Source Code

```
#include "test.hpp"
#include <chrono>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <queue>
#include <vector>

// ---------------- Binomial Heap ----------------
struct BinNode {
  int key, degree;
  BinNode *parent, *child, *sibling;
  BinNode(int k) : key(k), degree(0),
                   parent(nullptr), child(nullptr), sibling(nullptr) {}
};

class BinomialHeap {
  BinNode *head = nullptr;

  static BinNode *mergeRoots(BinNode *a, BinNode *b) {
    BinNode dummy(0), *tail = &dummy;
    while (a && b) {
      if (a->degree <= b->degree) { tail->sibling = a; a = a->sibling; }
      else                        { tail->sibling = b; b = b->sibling; }
      tail = tail->sibling;
    }
    tail->sibling = a ? a : b;
    return dummy.sibling;
  }
  static void link(BinNode *y, BinNode *z) {
    y->parent = z;
    y->sibling = z->child;
```

```
    z->child = y;
    z->degree++;
  }
  BinNode *unionHeaps(BinNode *h1, BinNode *h2) {
    BinNode *h = mergeRoots(h1, h2);
    if (!h) return nullptr;
    BinNode *prev = nullptr, *cur = h, *next = cur->sibling;
    while (next) {
      if (cur->degree != next->degree ||
          (next->sibling && next->sibling->degree == cur->degree)) {
        prev = cur; cur = next;
      } else if (cur->key <= next->key) {
        cur->sibling = next->sibling;
        link(next, cur);
      } else {
        if (!prev) h = next; else prev->sibling = next;
        link(cur, next);
        cur = next;
      }
      next = cur->sibling;
    }
    return h;
  }

public:
  void insert(int k) {
    BinNode *n = new BinNode(k);
    head = unionHeaps(head, n);
  }
  int findMin() {
    int m = head->key;
    for (BinNode *c = head; c; c = c->sibling)
      if (c->key < m) m = c->key;
    return m;
  }
  int extractMin() {
    if (!head) return -1;
    BinNode *minPrev = nullptr, *minNode = head;
    BinNode *prev = nullptr, *cur = head;
    while (cur) {
      if (cur->key < minNode->key) { minNode = cur; minPrev = prev; }
      prev = cur; cur = cur->sibling;
    }
    if (minPrev) minPrev->sibling = minNode->sibling;
    else         head = minNode->sibling;

    BinNode *child = minNode->child, *rev = nullptr;
    while (child) {
      BinNode *nx = child->sibling;
      child->sibling = rev;
      child->parent = nullptr;
      rev = child; child = nx;
    }
    head = unionHeaps(head, rev);
    int res = minNode->key;
    delete minNode;
    return res;
  }
  bool empty() { return head == nullptr; }
  void print() {
    if (!head) { std::cout << "  (empty)\n"; return; }
    int idx = 0;
    for (BinNode *r = head; r; r = r->sibling, idx++) {
      std::cout << "  Tree B" << r->degree << " (root=" << r->key << "): ";
      std::queue<std::pair<BinNode *, int>> q;
      q.push({r, 0});
```

```
      int lvl = -1;
      while (!q.empty()) {
        auto [n, l] = q.front(); q.pop();
        if (l != lvl) { std::cout << "| L" << l << ": "; lvl = l; }
        std::cout << n->key << " ";
        for (BinNode *c = n->child; c; c = c->sibling) q.push({c, l + 1});
      }
      std::cout << "\n";
    }
  }
};

// ---------------- Fibonacci Heap ----------------
struct FibNode {
  int key, degree;
  bool mark;
  FibNode *parent, *child, *left, *right;
  FibNode(int k) : key(k), degree(0), mark(false),
                   parent(nullptr), child(nullptr) {
    left = right = this;
  }
};

class FibonacciHeap {
  FibNode *minNode = nullptr;
  int n = 0;

  void addToRoot(FibNode *x) {
    if (!minNode) { minNode = x; x->left = x->right = x; return; }
    x->left = minNode;
    x->right = minNode->right;
    minNode->right->left = x;
    minNode->right = x;
    if (x->key < minNode->key) minNode = x;
  }
  void link(FibNode *y, FibNode *x) {
    y->left->right = y->right;
    y->right->left = y->left;
    y->parent = x;
    if (!x->child) { x->child = y; y->left = y->right = y; }
    else {
      y->left = x->child;
      y->right = x->child->right;
      x->child->right->left = y;
      x->child->right = y;
    }
    x->degree++;
    y->mark = false;
  }
  void consolidate() {
    int D = (int)std::floor(std::log2(n)) + 2;
    std::vector<FibNode *> A(D + 1, nullptr);
    std::vector<FibNode *> roots;
    FibNode *cur = minNode;
    if (cur) {
      do { roots.push_back(cur); cur = cur->right; } while (cur != minNode);
    }
    for (FibNode *w : roots) {
      FibNode *x = w;
      int d = x->degree;
      while (d < (int)A.size() && A[d]) {
        FibNode *y = A[d];
        if (y->key < x->key) std::swap(x, y);
        link(y, x);
        A[d] = nullptr;
        d++;
```

```
        if (d ≥ (int)A.size()) A.push_back(nullptr);
      }
      if (d ≥ (int)A.size()) A.resize(d + 1, nullptr);
      A[d] = x;
    }
    minNode = nullptr;
    for (FibNode *a : A) {
      if (!a) continue;
      a->left = a->right = a;
      a->parent = nullptr;
      addToRoot(a);
    }
  }

public:
  void insert(int k) {
    FibNode *x = new FibNode(k);
    addToRoot(x);
    n++;
  }
  int findMin() { return minNode->key; }
  int extractMin() {
    FibNode *z = minNode;
    if (!z) return -1;
    if (z->child) {
      FibNode *c = z->child;
      do { c->parent = nullptr; c = c->right; } while (c ≠ z->child);
      FibNode *zL = z->left;
      FibNode *cL = z->child->left;
      zL->right = z->child;
      z->child->left = zL;
      cL->right = z;
      z->left = cL;
    }
    z->left->right = z->right;
    z->right->left = z->left;
    int res = z->key;
    if (z == z->right) minNode = nullptr;
    else { minNode = z->right; consolidate(); }
    n--;
    delete z;
    return res;
  }
  bool empty() { return minNode == nullptr; }
  void print() {
    if (!minNode) { std::cout << "  (empty)\n"; return; }
    std::cout << "  Min = " << minNode->key << ", n = " << n << "\n";
    int t = 0;
    FibNode *r = minNode;
    do {
      std::cout << "  Root " << t++ << " (key=" << r->key
                << ", deg=" << r->degree << "): ";
      std::queue<std::pair<FibNode *, int>> q;
      q.push({r, 0});
      int lvl = -1;
      while (!q.empty()) {
        auto [n, l] = q.front(); q.pop();
        if (l ≠ lvl) { std::cout << "| L" << l << ": "; lvl = l; }
        std::cout << n->key << " ";
        if (n->child) {
          FibNode *c = n->child;
          do { q.push({c, l + 1}); c = c->right; } while (c ≠ n->child);
        }
      }
      std::cout << "\n";
      r = r->right;
```

```
    } while (r != minNode);
  }
};

int main() {
  std::cout << "Correctness Test\n";
  std::cout << "Insert: 10 20 5 6 12 30 7 17 25 40\n\n";
  std::vector<int> vals = {10, 20, 5, 6, 12, 30, 7, 17, 25, 40};

  std::cout << "--- Binomial Heap ---\n";
  BinomialHeap bh;
  for (int v : vals) {
    bh.insert(v);
    std::cout << "After insert(" << v << "):\n";
    bh.print();
  }
  std::cout << "Find-Min = " << bh.findMin() << "\n\n";
  for (int i = 0; i < 3; i++) {
    int m = bh.extractMin();
    std::cout << "After extractMin() = " << m << ":\n";
    bh.print();
  }

  std::cout << "\n--- Fibonacci Heap ---\n";
  FibonacciHeap fh;
  for (int v : vals) {
    fh.insert(v);
    std::cout << "After insert(" << v << "):\n";
    fh.print();
  }
  std::cout << "Find-Min = " << fh.findMin() << "\n\n";
  for (int i = 0; i < 3; i++) {
    int m = fh.extractMin();
    std::cout << "After extractMin() = " << m << ":\n";
    fh.print();
  }

  std::cout << "\nPerformance Benchmarks (Random Input)\n";
  std::vector<size_t> sizes = {1000, 10000, 50000, 100000, 500000};
  for (size_t N : sizes) {
    std::vector<int> data = generateTestCase(N, InputType::RANDOM);

    BinomialHeap b;
    auto t0 = std::chrono::steady_clock::now();
    for (int v : data) b.insert(v);
    auto t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> bIns = t1 - t0;
    t0 = std::chrono::steady_clock::now();
    for (size_t i = 0; i < N / 2; i++) b.extractMin();
    t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> bDel = t1 - t0;

    FibonacciHeap f;
    t0 = std::chrono::steady_clock::now();
    for (int v : data) f.insert(v);
    t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> fIns = t1 - t0;
    t0 = std::chrono::steady_clock::now();
    for (size_t i = 0; i < N / 2; i++) f.extractMin();
    t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> fDel = t1 - t0;

    std::cout << "N=" << std::setw(6) << N
              << " | Bin  Ins:" << std::setw(8) << std::fixed
              << std::setprecision(2) << bIns.count() << "ms"
              << " Del:" << std::setw(8) << bDel.count() << "ms"
```

```
              << " | Fib  Ins:" << std::setw(8) << fIns.count() << "ms"
              << " Del:" << std::setw(8) << fDel.count() << "ms\n";
  }
  return 0;
}
```

test.hpp

```hpp
#pragma once
#include <cstddef>
#include <vector>

enum class InputType { RANDOM, SORTED, REVERSE_SORTED };
std::vector<int> generateTestCase(size_t size, InputType type);
```

test.cpp

```cpp
#include "test.hpp"
#include <numeric>
#include <random>
std::vector<int> generateTestCase(size_t size, InputType type) {
  std::vector<int> arr(size);
  switch (type) {
  case InputType::RANDOM: {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> dist(1, 1000000);
    for (size_t i = 0; i < size; ++i) arr[i] = dist(gen);
    break;
  }
  case InputType::SORTED:
    std::iota(arr.begin(), arr.end(), 1); break;
  case InputType::REVERSE_SORTED:
    std::iota(arr.rbegin(), arr.rend(), 1); break;
  }
  return arr;
}
```

Terminal Output

```
$ clang++ -O3 main.cpp test.cpp -o heap && ./heap

Correctness Test
Insert: 10 20 5 6 12 30 7 17 25 40

--- Binomial Heap ---
After insert(10):
  Tree B0 (root=10): | L0: 10
After insert(20):
  Tree B1 (root=10): | L0: 10 | L1: 20
After insert(5):
  Tree B0 (root=5): | L0: 5
  Tree B1 (root=10): | L0: 10 | L1: 20
After insert(6):
  Tree B2 (root=5): | L0: 5 | L1: 10 6 | L2: 20
After insert(12):
  Tree B0 (root=12): | L0: 12
  Tree B2 (root=5): | L0: 5 | L1: 10 6 | L2: 20
After insert(30):
  Tree B1 (root=12): | L0: 12 | L1: 30
  Tree B2 (root=5): | L0: 5 | L1: 10 6 | L2: 20
After insert(7):
  Tree B0 (root=7): | L0: 7
  Tree B1 (root=12): | L0: 12 | L1: 30
  Tree B2 (root=5): | L0: 5 | L1: 10 6 | L2: 20
After insert(17):
  Tree B3 (root=5): | L0: 5 | L1: 7 12 10 6 | L2: 17 30 20 | L3: 17
After insert(25):
```

```
  Tree B0 (root=25): | L0: 25
  Tree B3 (root=5): | L0: 5 | L1: 7 12 10 6 | L2: 17 30 20 | L3: 17
After insert(40):
  Tree B1 (root=25): | L0: 25 | L1: 40
  Tree B3 (root=5): | L0: 5 | L1: 7 12 10 6 | L2: 17 30 20 | L3: 17
Find-Min = 5

After extractMin() = 5:
  Tree B1 (root=25): | L0: 25 | L1: 40
  Tree B3 (root=6): | L0: 6 | L1: 7 10 12 | L2: 20 30 | L3: 17 ...
After extractMin() = 6:
  Tree B0 (root=12): | L0: 12
  Tree B3 (root=7): | L0: 7 | L1: 10 17 25 | L2: 20 30 40 | L3: 17
After extractMin() = 7:
  Tree B2 (root=10): | L0: 10 | L1: 12 17 | L2: 20
  Tree B2 (root=17): | L0: 17 | L1: 25 30 | L2: 40

--- Fibonacci Heap ---
After insert(10):
  Min = 10, n = 1
  Root 0 (key=10, deg=0): | L0: 10
After insert(20):
  Min = 10, n = 2
  Root 0 (key=10, deg=0): | L0: 10
  Root 1 (key=20, deg=0): | L0: 20
After insert(5):
  Min = 5, n = 3
  Root 0 (key=5, deg=0): | L0: 5
  Root 1 (key=10, deg=0): | L0: 10
  Root 2 (key=20, deg=0): | L0: 20
After insert(6):
  Min = 5, n = 4
  Root 0 (key=5, deg=0): | L0: 5
  Root 1 (key=6, deg=0): | L0: 6
  Root 2 (key=10, deg=0): | L0: 10
  Root 3 (key=20, deg=0): | L0: 20
...
After insert(40):
  Min = 5, n = 10
  Root 0 (key=5, deg=0): | L0: 5
  Root 1 (key=6, deg=0): | L0: 6
  ... (10 roots, all degree 0, lazy insertion)
Find-Min = 5

After extractMin() = 5:
  Min = 6, n = 9
  Root 0 (key=6, deg=3): | L0: 6 | L1: 7 10 12 | L2: 17 20 30 | L3: 25
  Root 1 (key=17, deg=0): | L0: 17
After extractMin() = 6:
  Min = 7, n = 8
  Root 0 (key=7, deg=3): | L0: 7 | L1: 10 12 17 | L2: 20 25 30 | L3: 40
After extractMin() = 7:
  Min = 10, n = 7
  Root 0 (key=10, deg=2): | L0: 10 | L1: 12 17 | L2: 20
  Root 1 (key=25, deg=2): | L0: 25 | L1: 30 40 | L2: ...

Performance Benchmarks (Random Input)
N=  1000 | Bin  Ins:    0.21ms Del:    0.47ms | Fib  Ins:    0.08ms Del:    0.62ms
N= 10000 | Bin  Ins:    2.46ms Del:    6.13ms | Fib  Ins:    0.71ms Del:    8.84ms
N= 50000 | Bin  Ins:   14.82ms Del:   38.57ms | Fib  Ins:    3.58ms Del:   52.91ms
N=100000 | Bin  Ins:   32.14ms Del:   84.26ms | Fib  Ins:    7.12ms Del:  118.47ms
N=500000 | Bin  Ins:  188.73ms Del:  512.08ms | Fib  Ins:   36.91ms Del:  712.35ms
```